資料分析與機器學習
(Data Analysis and Machine Learning)

從沒有標籤的資料中找出
隱藏的結構
(依相似度探索未知群組)

Lecture 11 –
Clustering and Applications

教師：陳慶永

Outline

- Problem Statement
- Prototype-based Clustering
  - k-means
- Hierarchical Clustering
  - Agglomerative and divisive approaches
- Density-based Clustering
  - DBSCAN
- Graph-based Clustering
  - Spectral clustering
- Clustering in scikit-learn

Problem Statement

Clustering (分群 或 集群)

- Also called Cluster Analysis
- A category of unsupervised learning techniques
  - Allows us to discover *hidden structures* in data where we do not know the right answer upfront
- The goal is to find a natural grouping in data
  - Items in the same cluster are more similar to each other than to those from different clusters (群內相似，群間差異較大)

Distance and Feature Scaling

- How do we measure similarity between objects?
  - Define similarity as the *opposite* of the squared Euclidean distance between two points, $\boldsymbol{x}$ and $\boldsymbol{y}$, in m-dimensional space

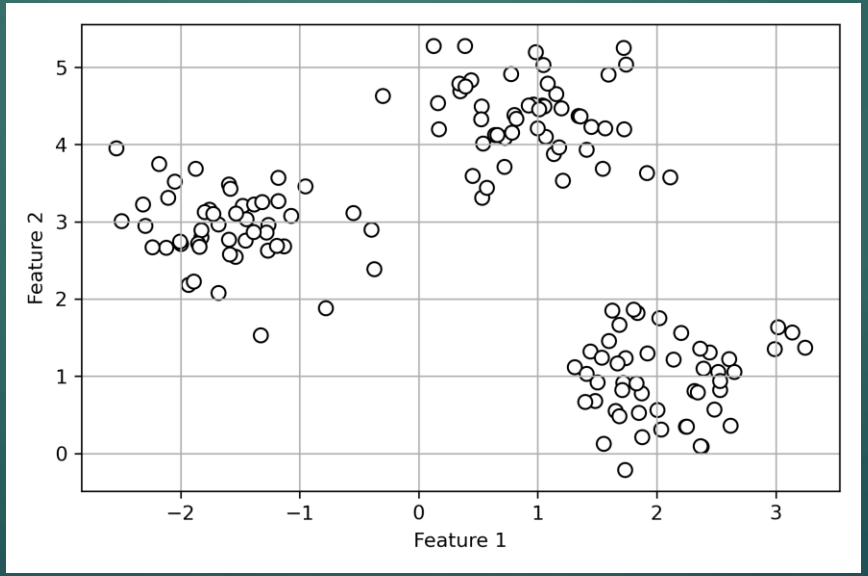
$$d(\boldsymbol{x}, \boldsymbol{y})^2 = \sum_{j=1}^{m} (x_j - y_j)^2 = \|\boldsymbol{x} - \boldsymbol{y}\|_2^2$$

(適合衡量連續特徵的距離)

  - The distance is dominated by features of large scales
    - Apply feature scaling to make sure that features are on the same scale

Applications

- Grouping of documents, music and movies by different topics
- Finding customers that share similar interests based on common purchase behaviors
- …



150 筆資料，2 個特徵

二維圖形只是為了適合示範，真實資料可能有很多維

Clustering Evaluation

- How to evaluate the performance of clustering
  - The main challenges in unsupervised learning is that we do not know the definitive answer
    - We don't have the ground-truth class labels in our dataset
- To quantify the quality of clustering, we need to use intrinsic metrics (內部指標) to compare the performance of different clustering models

Curse of Dimensionality

- Apply (unsupervised) dimensionality reduction techniques *prior to* performing clustering
  - It is particularly common to compress datasets down to two-dimensional subspaces
    - Allows us to visualize the clusters and assigned labels using two-dimensional scatterplots, which are particularly helpful for evaluating the results

Prototype-based Clustering

Prototype-based Clustering

- Each cluster is represented by a prototype
  - Either the centroid (質心)
    - Average of similar points with continuous features
  - Or the medoid (中心點)
    - The most representative point in the case of categorical features
- The most popular algorithm is k-means
  - Identify clusters with a spherical shape
  - *Have to* specify the number of clusters, k, a priori
    - An inappropriate choice for k can result in poor clustering performance

Centroid 是群內所有點的「平均值」(不一定存在於原始數據中)；而 Medoid 則是群內最具代表性的「一個真實數據點」

k-means Algorithm

- An *iterative* approach for minimizing the within-cluster sum of squared errors (SSE)
  - Also called cluster inertia

$$SSE = \sum_{i=1}^{n} \sum_{j=1}^{k} w^{(i,j)} \left\| \boldsymbol{x}^{(i)} - \boldsymbol{\mu}^{(j)} \right\|_2^2$$

$$w^{(i,j)} = \begin{cases} 1, & \text{if } \boldsymbol{x}^{(i)} \in j \\ 0, & \text{otherwise} \end{cases}$$

sample index

cluster index

Centroid for cluster *j*
(第 *j* 群的平均特徵向量)

  - An intrinsic metric to evaluate clustering performance

目標是降低群內平方距離總和 (SSE)

k-means Procedure

1. Randomly pick k centroids from the examples as initial cluster centers
2. Assign each example to the nearest centroid, $\mu^{(j)}, j \in \{1, \ldots, k\}$
3. Move the centroids to the center of the examples that were assigned to it
4. Repeat *steps 2* and *3* until the cluster assignments do not change or a user-defined tolerance or maximum number of iterations is reached

- 指定 k，隨機初始化 k 個質心
- 將每筆資料指派給最近的質心
- 以群內平均值更新質心
- 重複指派與更新，直到停止條件成立

A Small Example

- 4 筆一維資料待分群：1、2、8、9
- 指定 $k = 2$，初始質心：$\mu_1 = 1$、$\mu_2 = 8$
- 第一次指派：{1, 2} 與 {8, 9} (SSE = 2)
- 更新質心：1.5 與 8.5 (SSE = $4 \times 0.5^2 = 1$)

k-means in scikit-learn

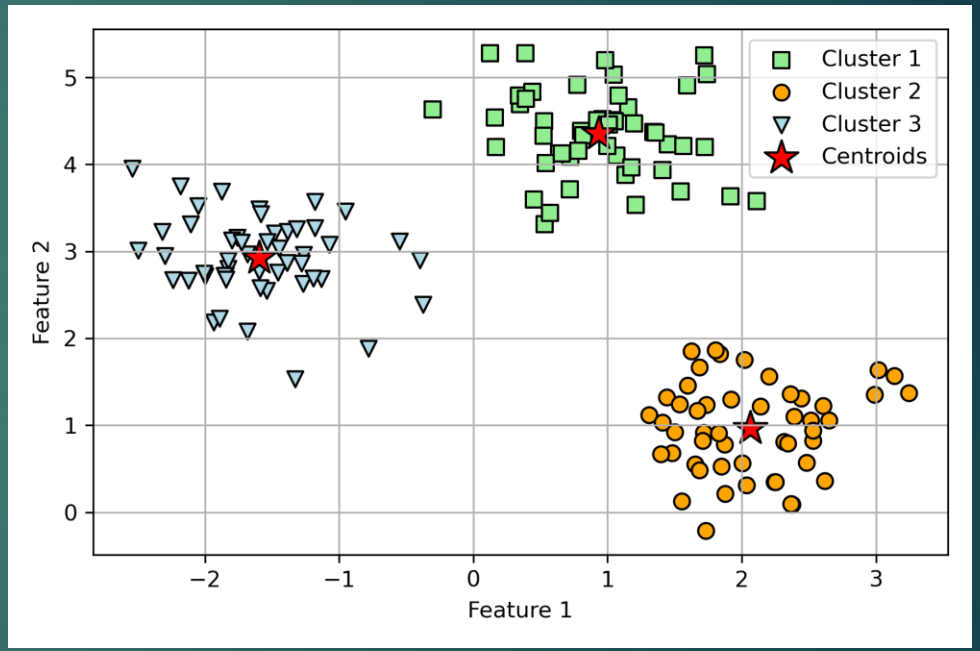
- sklearn.cluster.KMeans

```
from sklearn.cluster import KMeans
km = KMeans(n_clusters=3,
            init='random',
            n_init=10,
            max_iter=300,
            tol=1e-04,
            random_state=0)
y_km = km.fit_predict(X)
```

須指定群數 k

k-means++

不同初始化的執行次數

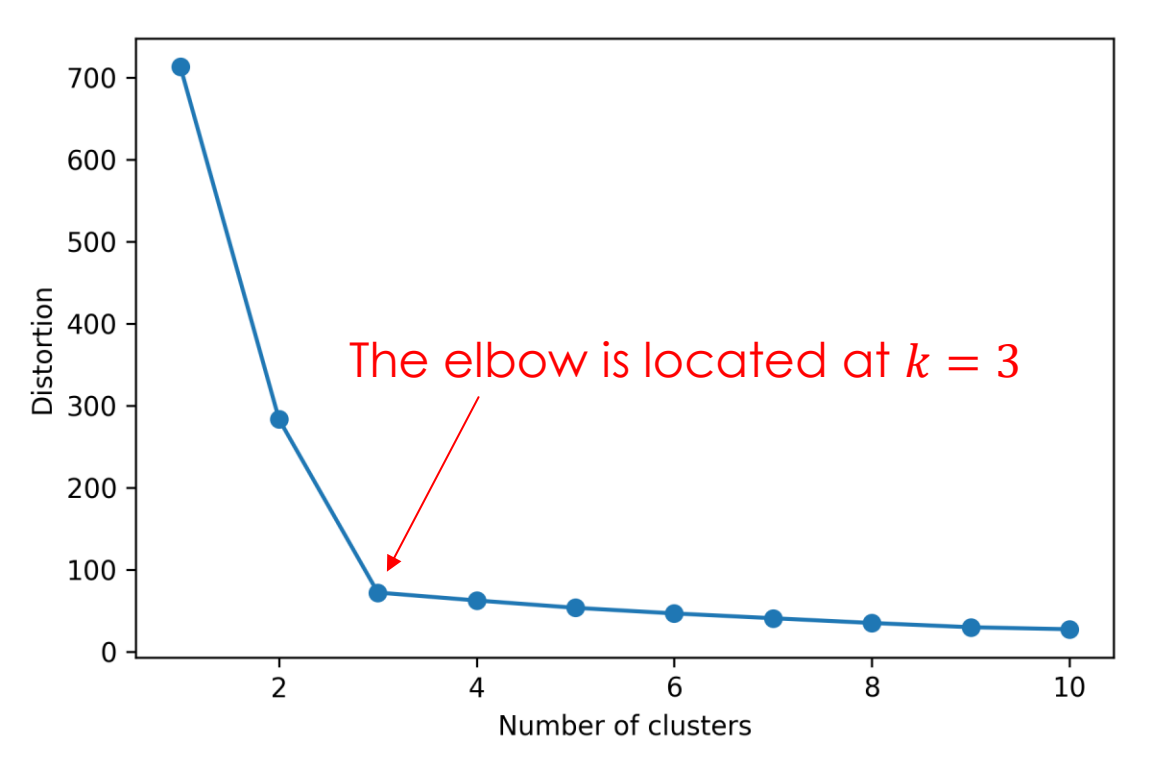


Run the algorithm n_init times independently and choose the final model as the one with the lowest SSE

Elbow Method (手肘法)

- A graphical tool to estimate the optimal number of clusters, k, based on the within-cluster SSE

Within-cluster SSE



比較不同 k 的群內 SSE，觀察 SSE 下降幅度開始趨緩的位置

Hierarchical Clustering

Hierarchical Clustering

- Organize clusters as dendrograms (樹狀圖)
  - Visualizations by a hierarchical tree
  - Do *not* need to specify the number of clusters upfront
- Two main approaches
  - ✓ Agglomerative hierarchical clustering (聚合式階層分群法)
    - Bottom-up (每點一群，逐步合併)
  - ✗ Divisive hierarchical clustering (分裂式階層分群法)
    - Top-down (全部一群，逐步分割)

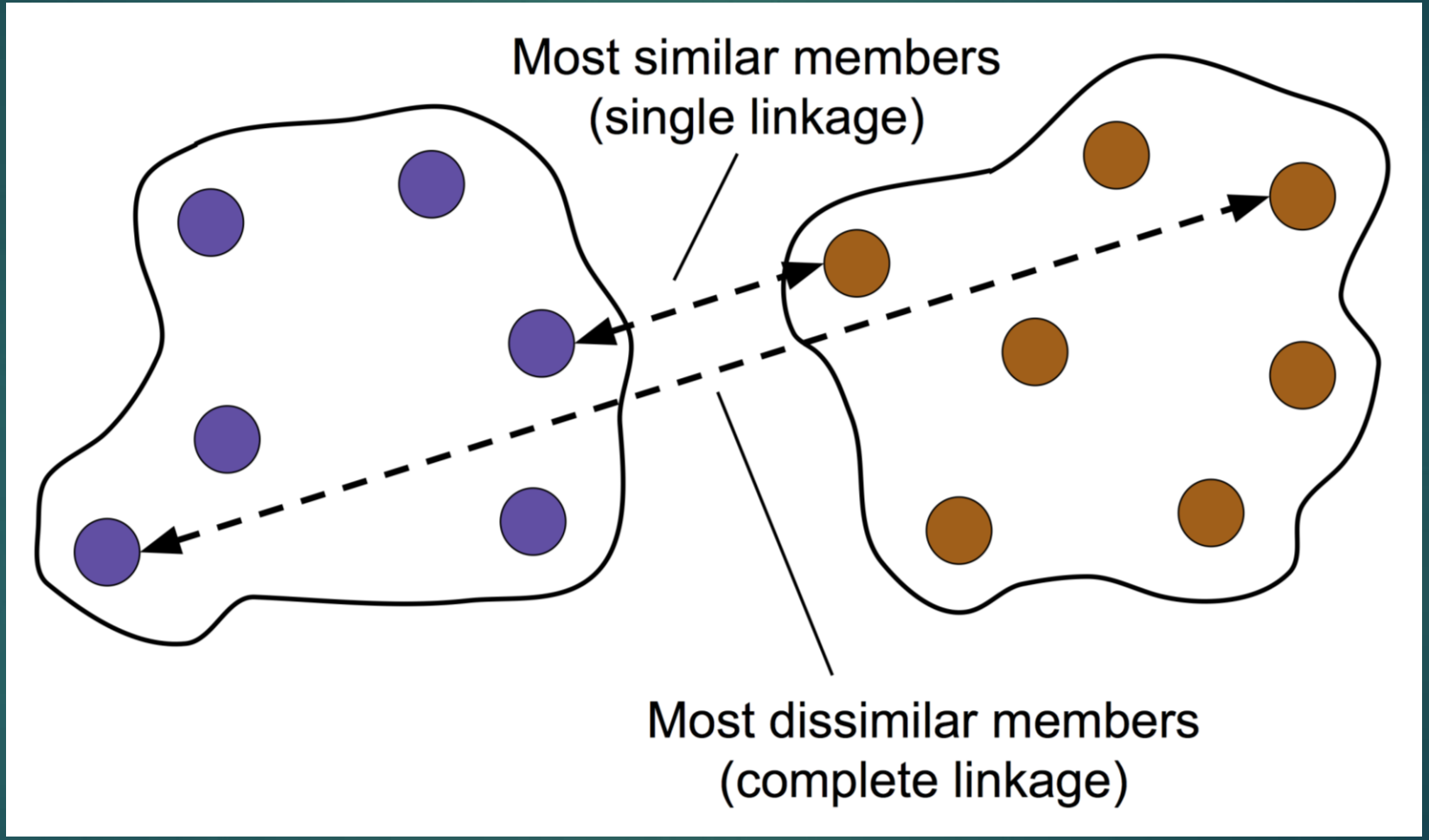
分裂式階層分群法運算繁複且早期分割錯誤難以彌補，實務應用中遠不如聚合式階層分群法

Linkage Criteria (連結準則)

- Single linkage (兩群之間最小的跨群點距離)
  - Compute the distances between the most similar members for each pair of clusters
  - Merge the two clusters for which the distance between the most similar members is the smallest
- Complete linkage (兩群之間最大的跨群點距離)
  - Similar to single linkage but compare the most dissimilar members to perform the merge

Other commonly used algorithms for agglomerative hierarchical clustering include average linkage and Ward's linkage

Linkage Criteria (連結準則)



Agglomerative Clustering Procedure

1. Compute a pair-wise distance matrix of all samples
2. Represent each data point as a singleton cluster
3. Merge the two closest clusters based on the distance between the most dissimilar (distant) members (using complete linkage)
4. Update the cluster linkage matrix (連結矩陣)
5. Repeat steps 2-4 until one single cluster remains

每次選擇群間 complete distance (最遠跨群成員間距離) 最小的一對合併

A Small Example

- 5 筆 (IDs 0-4) 三維 (X, Y, Z) 資料待分群

	X	Y	Z
ID_0	6.964692	2.861393	2.268515
ID_1	5.513148	7.194690	4.231065
ID_2	9.807642	6.848297	4.809319
ID_3	3.921175	3.431780	7.290497
ID_4	4.385722	0.596779	3.980443

Pair-wise distances

	ID_0	ID_1	ID_2	ID_3	ID_4
ID_0	0.000000	4.973534	5.516653	5.899885	3.835396
ID_1	4.973534	0.000000	4.347073	5.104311	6.698233
ID_2	5.516653	4.347073	0.000000	7.244262	8.316594
ID_3	5.899885	5.104311	7.244262	0.000000	4.382864
ID_4	3.835396	6.698233	8.316594	4.382864	0.000000

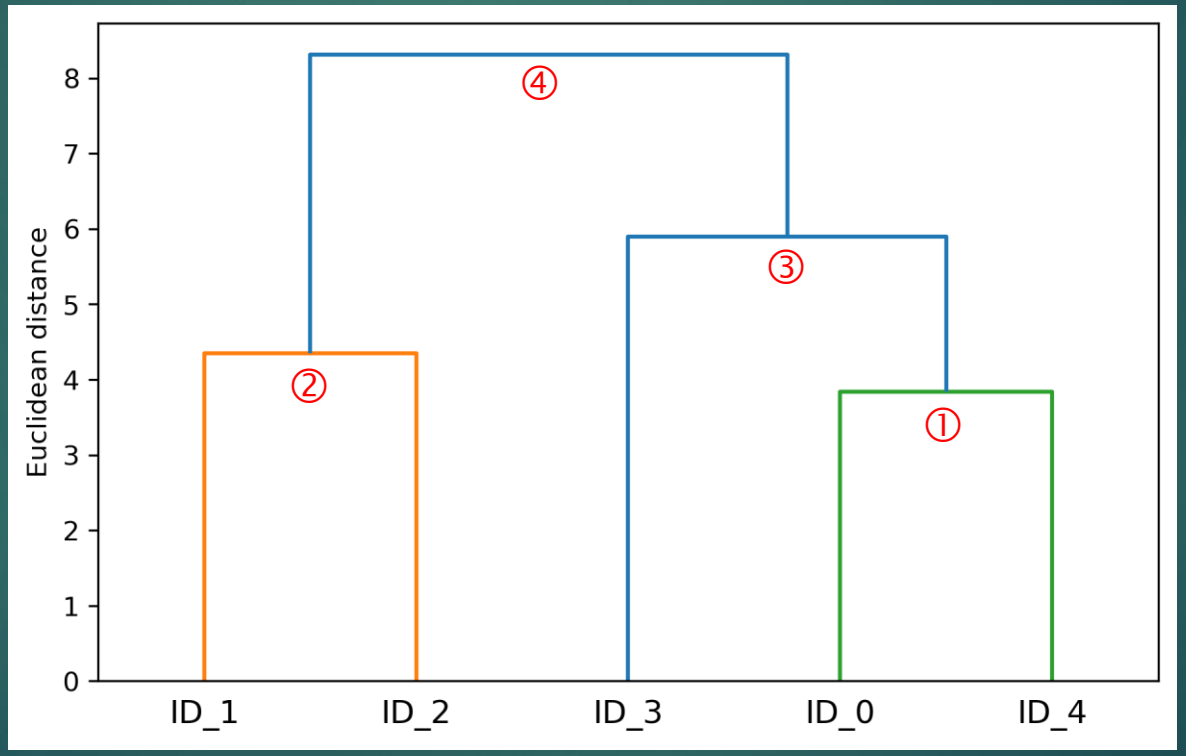
A Small Example

- Linkage matrix (連結矩陣)
  - 每列記錄一次合併，共 $n-1$ 列
  - 欄 1、2：被合併的群 ID
  - 欄 3：這次合併的 linkage distance
  - 欄 4：新群包含的原始樣本數

	row label 1	row label 2	distance	no. of items in clust.
cluster 1	0.0	4.0	3.835396	2.0
cluster 2	1.0	2.0	4.347073	2.0
cluster 3	3.0	5.0	5.899885	3.0
cluster 4	6.0	7.0	8.316594	5.0

A Small Example

- Dendrogram



葉節點代表原始樣本，合併高度代表 linkage distance。在某高度畫一條水平線切樹，可取得一個分群結果

Agglomerative Clustering in scikit-learn

- sklearn.cluster.AgglomerativeClustering
  - Allows us to choose the number of desired clusters
  - Useful if we want to prune the hierarchical cluster tree

```
>>> from sklearn.cluster import AgglomerativeClustering
>>> ac = AgglomerativeClustering(n_clusters=3,
...                              metric='euclidean',
...                              linkage='complete')
>>> labels = ac.fit_predict(X)
>>> print(f'Cluster labels: {labels}')
Cluster labels: [1 0 0 2 1]
```

可任意指定切樹後的群數

連結準則

Density-based Clustering

Density-based Clustering

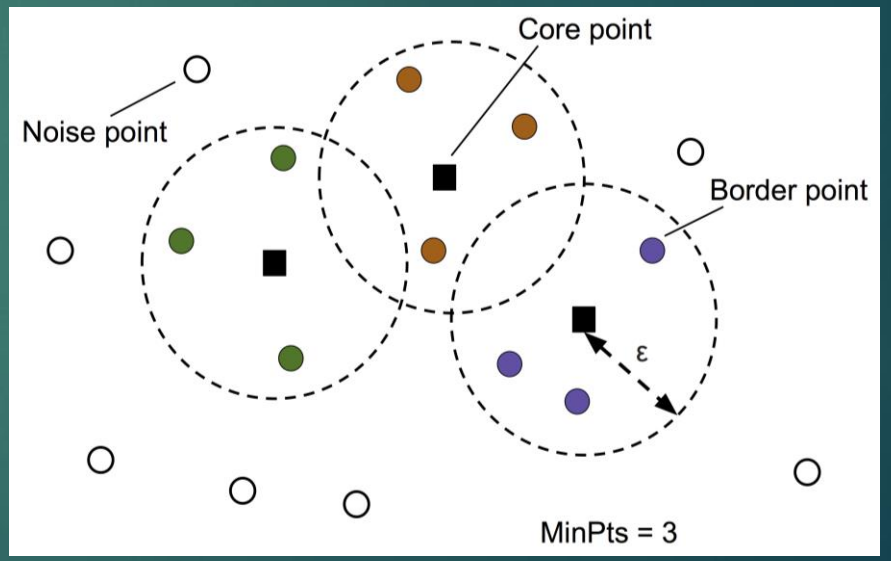
- Assign cluster labels based on dense regions of points (density) (以局部密度連接群組)
  - Do not make assumptions about spherical clusters (like k-means)
  - Do not partition the dataset into hierarchies that require a manual cut-o point

DBSCAN

- Density-based spatial clustering of applications with noise (DBSCAN) (基於密度的雜訊空間分群演算法)
  - The notion of density is defined as the number of points within a specified radius ε (鄰域半徑內的樣本數)
  - A label is assigned to each data point using the criteria:
    - Core point (核心): At least a specified number (MinPts) of neighboring points fall within the specified radius ε
    - Border point (邊界): Has fewer neighbors than MinPts within ε, but lies within the ε radius of a core point
    - Noise point (雜訊): All other points that are neither core nor border points

DBSCAN

- Hyperparameters: ε (鄰域半徑) and MinPts (鄰域樣本數門檻)
  - Core (鄰域樣本數達標)
  - Border (未達標，但位於某核心點鄰域)
  - Noise (不屬於核心或邊界點)



Finding a good combination of MinPts and ε can be problematic if the density differences in the dataset are relatively large

DBSCAN

- After labeling the points as core, border, or noise, the DBSCAN algorithm can be summarized in two simple steps:
  1. Form a separate cluster for each core point or connected group of core points (Core points are connected if they are no farther away than ε)
  2. Assign each border point to the cluster of its corresponding core point

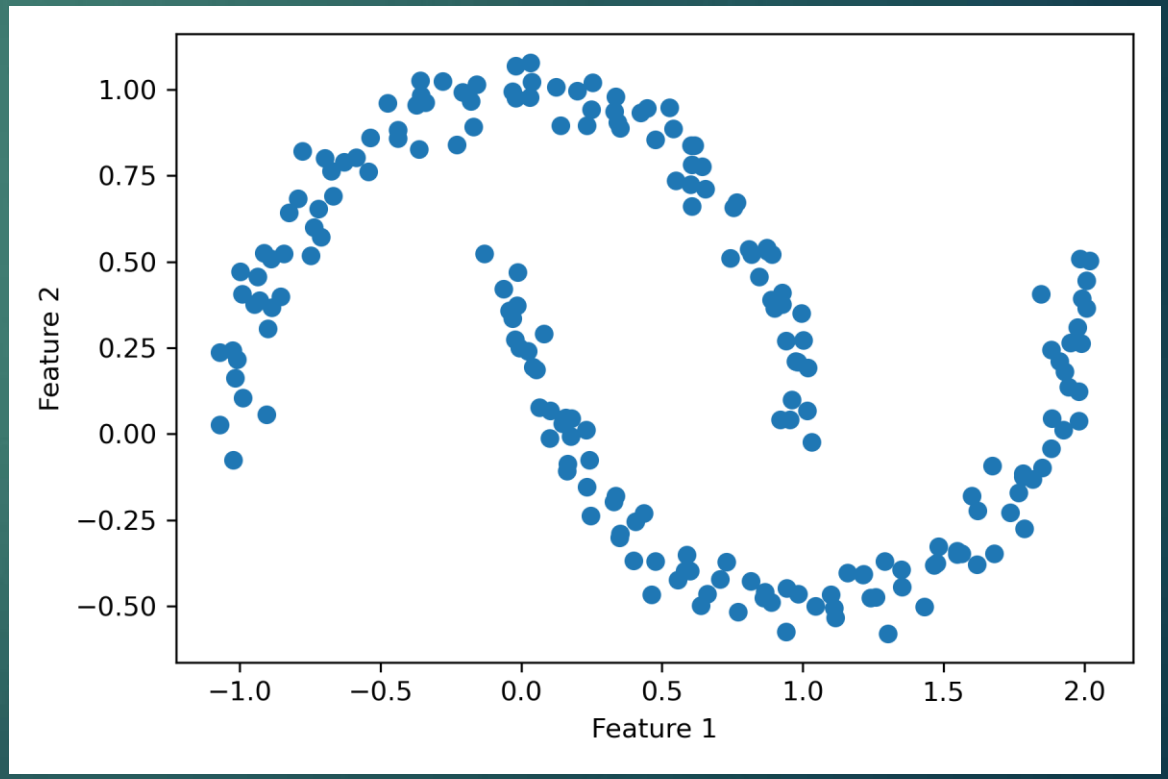
先連接彼此鄰近的核心點，沿核心點鏈持續擴張，再把邊界點附加到可到達的群

DBSCAN doesn't necessarily assign each point to a cluster but is capable of removing noise points

A Small Example

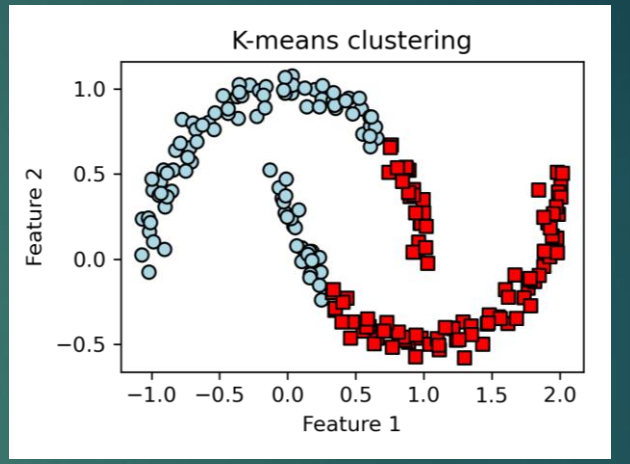
- 200 筆二維資料待分群
  - 實際上形成兩個半月 (half-moon-shaped)

```
>>> from sklearn.datasets import make_moons
>>> X, y = make_moons(n_samples=200,
...                   noise=0.05,
...                   random_state=0)
>>> plt.scatter(X[:, 0], X[:, 1])
>>> plt.xlabel('Feature 1')
>>> plt.ylabel('Feature 2')
>>> plt.tight_layout()
>>> plt.show()
```

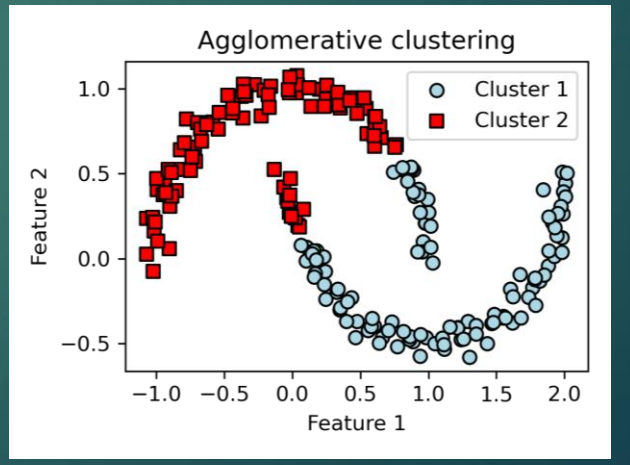


A Small Example

- k-means

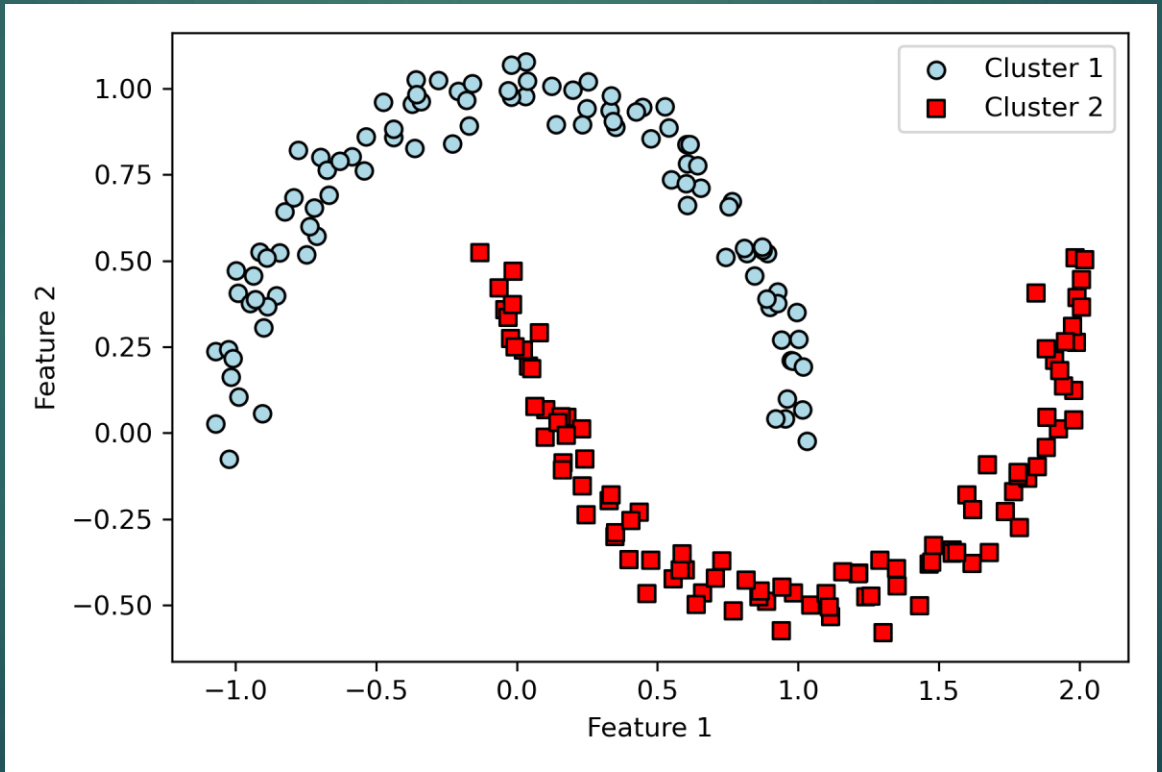


- Agglomerative Clustering



A Small Example

- DBSCAN ($\varepsilon = 0.2,\ \mathrm{MinPts} = 5$)
  - Cluster data of arbitrary shapes



DBSCAN in scikit-learn

- sklearn.cluster.DBSCAN

```
>>> from sklearn.cluster import DBSCAN
>>> db = DBSCAN(eps=0.2,
...             min_samples=5,
...             metric='euclidean')
>>> y_db = db.fit_predict(X)
```

ε

MinPts

Cluster labels:
Noisy samples are given the label -1;
Non-negative integers indicate cluster membership

Clustering in scikit-learn

Mission

- Clustering of complex dataset using different algorithms
  - k-means
  - Agglomerative clustering
  - DBSCAN
- Apply it to the 64-dimensional dataset, Digits (低解析度手寫數字資料集)
  - sklearn.datasets.load_digits()
  - Low-resolution handwrit ten digits (0-9)
- Sample code: 11_1.py

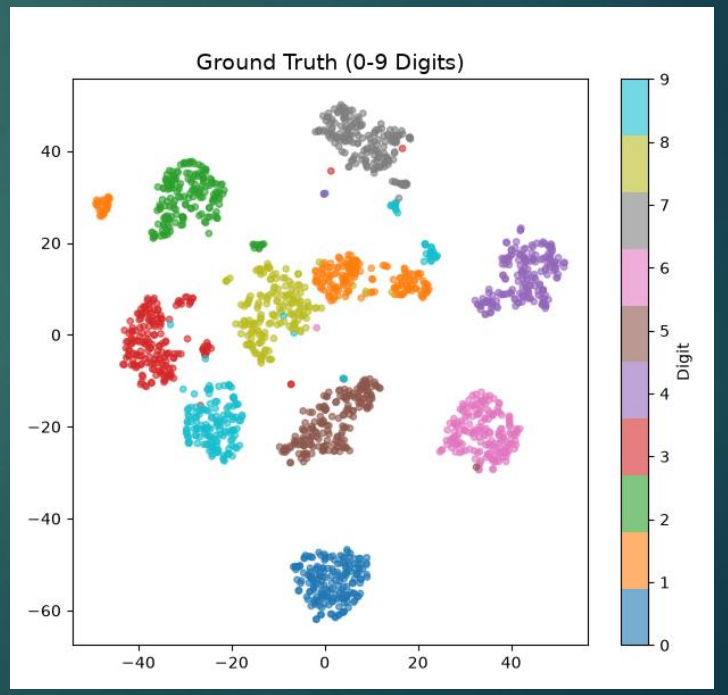
Dimensionality Reduction

- Reduced to 2-dimensional features using the t-SNE
  - For visualization of the clustering results
  - The ground truth is only used for evaluation

```python
import matplotlib.pyplot as plt
import numpy as np
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN
from sklearn.datasets import load_digits
from sklearn.manifold import TSNE
from sklearn.metrics import adjusted_rand_score

# 1. 資料準備與降維
digits = load_digits()
X, y = digits.data, digits.target

# 使用 t-SNE 將 64 維降至 2 維（便於視覺化與分群比較）
tsne = TSNE(n_components=2, init='pca', random_state=42)
X_tsne = tsne.fit_transform(X)
```



Clustering

```
16    # 2. 實作三種分群演算法（基於 t-SNE 降維後的 2D 數據）
17    # K-Means（設定 10 個群集）
18    kmeans = KMeans(n_clusters=10, random_state=42, n_init='auto')
19    y_kmeans = kmeans.fit_predict(X_tsne)
20
21    # Agglomerative Clustering（設定 10 個群集）
22    agg = AgglomerativeClustering(n_clusters=10, linkage="ward")
23    y_agg = agg.fit_predict(X_tsne)
24
25    # DBSCAN（調整參數以適應 t-SNE 的密度分佈）
26    # t-SNE 後的數據間距較大，這裡將 eps 設為大約 4.5 到 5.5 之間
27    dbscan = DBSCAN(eps=4.8, min_samples=15)
28    y_dbscan = dbscan.fit_predict(X_tsne)
29
```

Clustering Evaluation

- Adjusted Rand Index (ARI)
  - 評估分群結果與「真實標籤 (Ground Truth)」(或另一組分群結果) 之間相似度的外部評估指標

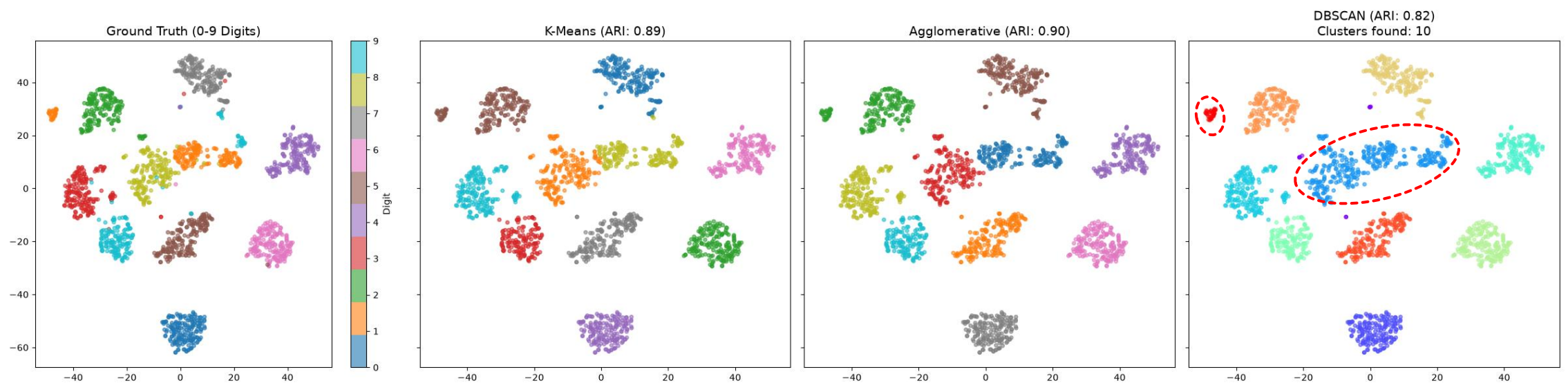
```
30    # 3. 計算評估指標 (Adjusted Rand Index, ARI)
31    ari_kmeans = adjusted_rand_score(y, y_kmeans)
32    ari_agg = adjusted_rand_score(y, y_agg)
33    ari_dbscan = adjusted_rand_score(y, y_dbscan)
34
35    # 4. 數據結果列印
36    print("\n--- ARI 比較 (越接近 1 越完美) ---")
37    print(f"K-Means ARI                : {ari_kmeans:.4f}")
38    print(f"Agglomerative (Ward) ARI : {ari_agg:.4f}")
39    print(f"DBSCAN ARI                 : {ari_dbscan:.4f}")
```

```
K-Means ARI                : 0.8891
Agglomerative (Ward) ARI : 0.9002
DBSCAN ARI                 : 0.8232
```

Visualization of Clusters

```
41  # 5. 繪圖視覺化比較
42  fig, axes = plt.subplots(1, 4, figsize=(24, 6), sharex=True, sharey=True)
43  cmap_discrete = plt.get_cmap("tab10", 10)
44
45  # 子圖 1: Ground Truth (真實標籤)
46  scatter0 = axes[0].scatter(X_tsne[:, 0], X_tsne[:, 1], c=y, cmap=cmap_discrete, alpha=0.6, s=15)
47  axes[0].set_title("Ground Truth (0-9 Digits)", fontsize=14)
48  fig.colorbar(scatter0, ax=axes[0], ticks=range(10), label="Digit")
49
50  # 子圖 2: K-Means
51  axes[1].scatter(X_tsne[:, 0], X_tsne[:, 1], c=y_kmeans, cmap=cmap_discrete, alpha=0.6, s=15)
52  axes[1].set_title(f"K-Means (ARI: {ari_kmeans:.2f})", fontsize=14)
53
54  # 子圖 3: Agglomerative
55  axes[2].scatter(X_tsne[:, 0], X_tsne[:, 1], c=y_agg, cmap=cmap_discrete, alpha=0.6, s=15)
56  axes[2].set_title(f"Agglomerative (ARI: {ari_agg:.2f})", fontsize=14)
57
58  # 子圖 4: DBSCAN (雜訊會被標記為 -1，在圖中會呈現不同的顏色)
59  axes[3].scatter(X_tsne[:, 0], X_tsne[:, 1], c=y_dbscan, cmap="rainbow", alpha=0.6, s=15)
60  n_clusters_db_ = len(set(y_dbscan)) - (1 if -1 in y_dbscan else 0)
61  axes[3].set_title(f"DBSCAN (ARI: {ari_dbscan:.2f})\nClusters found: {n_clusters_db_}", fontsize=14)
62
63  plt.tight_layout()
64  plt.show()
```

Visualization of Clusters



Discussions

- 經過 t-SNE 降維後，手寫數字會在 2D 空間中凝聚成 10 個相對明顯、但形狀不規則的「島嶼」群集
- K-Means
  - 表現優秀 (雖然數字群集形狀不完全是圓形，但質心分佈明確)
- Agglomerative
  - 表現優秀 (Ward 連結準則會尋找群內 SSE 最小的合併方式)
- DBSCAN
  - 表現高度依賴超參數
  - 優勢：如果超參數調整得當，能「自動」發現剛好有 10 群，不需要人工指定
  - 缺點：如果島嶼與島嶼之間有少數「橋接點」連在一起，兩個群集會被誤判為同一個
  - 有些落單的點會被直接定義為雜訊 (標籤 -1)

Thank you